Acceso a Datos

2º DAM - Curso 2021-2022

Acceso a Bases de Datos NoSQL. MongoDB

¿Qué es MongoDB

MongoDB [http://mongodb.com] es una base de datos orientada a documentos, clasificada como un SGBD NoSQL. Utiliza documentos JSON para crear el esquema de la base de datos.

Estructura de una Base de Datos

Como se ha comentado, *MongoDB* almacena la información y forma el esquema de la base de datos mediante el uso de documentos *JSON*, tal y como se muestra en la siguiente figura:

Figure 1: Documento JSON en MongoDB

Así, el esquema en MongoDB es totalmente flexible puesto que no obliga a diseñar un esquema o modelo antes de poder comenzar a registrar información. De esa forma es mucho más fácil hacer mapear los documentos con objetos en nuestra aplicación, puesto que es fácil adaptarnos a los cambios que estos últimos puedan sufrir

Puesta en marcha de MongoDB

Una vez descargado MongoDB [http://www.mongodb.org], para ponerlo en marcha, tendremos que crear la ruta donde queremos que se almacenen nuestras bases de datos. Y a continuación, lanzar el servidor ejecutando el siguiente comando:

Hay que tener en cuenta que, por defecto, *MongoDB* escuchará en el puerto 27017:

- En Windows

```
c:\Users\Santi\> mongod --dbpath=ruta_base_de_datos
```

- En Linux/OSX

```
santi@zenbook:$ ./mongod --dbpath=ruta_base_de_datos
```

Además, *MongoDB* dispone de una consola cliente que podemos lanzar ejecutando el comando `mongo`, que intentará conectar directamente con un servidor ubicado en el propio equipo y con el puerto por defecto, el 27017. También podemos especificar esos parámetros desde la línea de comandos al ejecutarlo (junto con usuario y contraseña si fuera necesario también):

- En Windows

```
c:\Users\Santi\> mongo --host 192.168.100.23 --port 27017 -u usuario -p password
```

- En Linux/OSX

```
santi@zenbook:$ ./mongo --host 192.168.100.23 --port 27017 -u usuario -p password
```

Una vez ejecutado entraremos en la consola de *MongoDB* donde, entre otros, tenemos los siguientes comandos disponibles:

- Ver las bases de datos existentes

```
> show dbs
biblioteca   0.203GB
ejemplo      0.203GB
local        0.078GB
```

- Conectar/Crear una base de datos

```
> use biblioteca
switched to db biblioteca
```

- Ver las colecciones dentro de una base de datos

```
> show collections
libros
system.indexes
```

- Eliminar una base de datos

```
> db.dropDatabase()
{ "dropped" : "biblioteca", "ok" : 1 }
```

- Mostrar ayuda

```
> help
        db.help()                    help on db methods
        db.mycoll.help()             help on collection methods
        sh.help()                    sharding helpers
        rs.help()                    replica set helpers
        help admin                   administrative help
        help connect                 connecting to a db help
. . .
. . .
```

Conectar con MongoDB

Para empezar a trabajar con MongoDB desde Java, primero tendremos que hacernos con el Driver. Podéis encontrar su enlace de descargar en la sección Software Necesario [http://datos.codeandcoke.com/extra:software] de los apuntes.

Conectar con la Base de Datos MongoDB desde Java

```
MongoClient mongoClient = new MongoClient();
MongoDatabase db = mongoClient.getDatabase("basededatos");
```

Desconectar de la Base de Datos

```
mongoClient.close()
```

Hay que tener en cuenta que para poder conectarnos con *MongoDB* primero tendremos que arrancar el servidor de la forma que se indica un poco más arriba en la puesta en marcha.

Utilizar CodecRegistry para mapear POJOs a Documentos

También podemos, si hemos creado un *CodecRegistry* a la hora de conectar con MongoDB, trabajar directamente con los POJO para insertar, modificar o leer documentos en la base de datos:

```
. . .
CodecRegistry pojoCodecRegistry = CodecRegistries.fromRegistries(MongoClient.getDefaultCodecRegistry(),
    CodecRegistries.fromProviders(PojoCodecProvider.builder().automatic(true).build()));
MongoClient cliente = new MongoClient("localhost",
    MongoClientOptions.builder().codecRegistry(pojoCodecRegistry).build());
MongoDatabase db = cliente.getDatabase(NOMBRE_BASEDATOS);
. . .
```

A través del *CodecRegistry* los objetos Java se mapearán automáticamente como documentos a la hora de insertarlos y modificarlos en la colección correspondiente, y también serán mapeados automáticamente a objetos Java desde la colección MongoDB cuando hagamos una lectura.

Más adelante, en los ejemplos de operaciones básicas de inserción, modificación y lectura, se muestran ejemplos de cómo realizar esas operaciones también en el caso de que se haya optado por instanciar y asociar a nuestra conexión un *CodecRegistry*.

Operaciones básicas

MongoDB proporciona varias vías para ejecutar cada una de las operaciones CRUD [1] (Create, Read, Update, Delete) que podemos realizar en todo SGBD.

Operaciones de creación

- *db.collection.insert()*
- *db.collection.insertOne()*
- *db.collection.insertMany()*

Ejemplo desde línea de comandos

```
db.libros.insert(                    // colección
  {                                  // documento
    titulo: "Secuestrado",
    descripcion: "Las aventuras de David Balfour",
    autor: "Robert Louis Stevenson",
    fecha: "2/5/2002",
    disponible: "true"
  }
)
```

Ejemplo desde Java

```
Document documento = new Document()
         .append("titulo", libro.getTitulo())
         .append("descripcion", libro.getDescripcion())
         .append("autor", libro.getAutor())
         .append("fecha", libro.getFecha())
         .append("disponible", libro.getDisponible()));
db.getCollection("libros").insertOne(documento);
```

Si hemos instanciado un *CodecRegistry* con nuestra conexión a MongoDB (como se explica en el apartado de Conectar con MongoDB [https://datos.codeandcoke.com/apuntes:mongodb#conectar_con_mongodb]), suponiendo que tenemos un objeto *coche* de la clase *Coche* que queremos insertar como documento en una colección MongoDB, podríamos hacer lo siguiente:

```
MongoCollection<Coche> coleccionCoches = db.getCollection("coches", Coche.class);
coleccionCoches.insertOne(coche);
```

Operaciones de lectura

- *db.collection.find()*

Ejemplo desde consola

```
db.usuarios.find(                                  // colección
  {autor: {$eq: "Robert Louis Stevenson"}},        // criterio
  {titulo: 1, descripcion: 1}                      // projección
).limit(10)                                        // modificador del cursor
```

Ejemplo desde Java [2)]

```
Document documento = new Document("autor", "Robert Louis Stevenson");
FindIterable findIterable = db.getCollection("libros")
                .find(documento)
                .limit(10);

List<Libro> libros = new ArrayList<Libro>();
Libro libro = null;
Iterator<Document> iter = findIterable.iterator();
while (iter.hasNext()) {
  Document documento = iter.next();
  libro = new Libro();
  libro.setId(documento.getObjectId("_id"));
  libro.setTitulo(documento.getString("titulo"));
  libro.setDescripcion(documento.getString("descripcion"));
  libro.setAutor(documento.getString("autor"));
  libro.setFecha(documento.getDate("fecha"));
  libro.setDisponible(documento.getBoolean("disponible", false));
  libros.add(libro);
}
```

Utilizando *CodecRegistry* (como se explica en el apartado de Conectar con MongoDB [https://datos.codeandcoke.com/apuntes:mongodb#conectar_con_mongodb]) para mapear automáticamente los POJO como documentos de una colección de MongoDB, también podemos leer:

```
. . .
MongoCollection<Coche> coleccionCoches = db.getCollection("coches", Coche.class);
List<Coche> coches = coleccionCoches.find().into(new ArrayList<Coche>());
. . .
```

De esa manera obtenemos, mapeando automáticamente a través del *CodecRegistry*, una colección de objetos *Coche* casi directamente de una colección MongoDB.

Operaciones de modificacion

- *db.collection.update()*
- *db.collection.updateOne()*
- *db.collection.updateMany()*
- *db.collection.replaceOne()*

Ejemplo desde consola

```
db.usuarios.update(                              // colección
  { titulo: { $eq: "Secuestrado"}},              // criterio
  { $set: {autor: "Robert Louis Stevenson"}}      // modificación
)
```

Ejemplos desde Java

```
// Modifica un campo específico de un documento
db.getCollection("libros").updateOne(new Document("titulo", "Secuestrado"),
         new Document("$set", new Document("autor", "Robert Louis Stevenson"));

// Reemplaza un documento completo
db.getCollection("libros").replaceOne(new Document("_id", libro.getId()),
         new Document()
         .append("titulo", libro.getTitulo())
         .append("descripcion", libro.getDescripcion())
         .append("autor", libro.getAutor())
         .append("fecha", libro.getFecha())
         .append("disponible", libro.getDisponible()));
```

De nuevo, utilizando *CodecRegistry* (como se explica en el apartado de Conectar con MongoDB [https://datos.codeandcoke.com/apuntes:mongodb#conectar_con_mongodb]) para el mapeo automático de los documentos en objetos Java (POJO), podemos modificar un documento de MongoDB pasándole directamente el objeto Java. En el ejemplo siguiente, tenemos el objeto con todos los campos ya modificados (excepto el id, que nunca tocaremos) y, utilizando el id como filtro, pasamos el objeto que será mapeado al documento que reemplazará al que haya actualmente en la base de datos:

```
. . .
MongoCollection<Coche> coleccionCoches = db.getCollection("coches", Coche.class);
coleccionCoches.replaceOne(eq("_id", coche.get_id()), coche);
. . .
```

Operaciones de borrado

- *db.collection.remove()*
- *db.collection.deleteOne()*
- *db.collection.deleteMany()*

Ejemplo desde consola

```
db.usuarios.remove(                  // colección
  { poblacion: "Zaragoza"}           // criterio
)
```

Ejemplo desde Java:

```
db.getCollection("libros").deleteOne(new Document("titulo", titulo));
```

También podemos hacerlo directamente sobre la colección, utilizando de nuevo un *CodeRegistry* (como se explica en el apartado de Conectar con MongoDB [https://datos.codeandcoke.com/apuntes:mongodb#conectar_con_mongodb]). En el ejemplo siguiente, tenemos el objeto cuyo documento queremos eliminar de la base de datos, y utilizamos el id como filtro para localizarlo y borrarlo de la colección:

```
. . .
MongoCollection<Coche> coleccionCoches = db.getCollection("coches", Coche.class);
coleccionCoches.deleteOne(eq("_id", coche.get_id()));
. . .
```

Búsquedas

En este apartado vamos a ver diferentes formas de realizar búsqueda de documento sobre una Base de Datos MongoDB.

Para realizar búsquedas se utiliza el método *find()*, que sería el equivalente a *SELECT* que utilizamos en el lenguaje SQL en Bases de Datos Relacionales. A ese método le podemos pasar un documento que haga de patrón para buscar en la colección todos aquellos que coincidan con él, como se puede ver en el ejemplo que se ha hecho anteriormente. Pero también se pueden pasar una serie de métodos que permiten establecer condiciones muy al estilo de las que se pasaban en las clausulas *WHERE* de las sentencias *SQL*.

- *find(eq("titulo", "Secuestrado"))* - Buscará todos los documentos que tengan como titulo el valor 'Secuestrado'. También se puede utilizar para buscar valores dentro de un array en el documento de una colección
- *find(gt("paginas", 50))* - Buscará todos los documentos que tengan más de 50 páginas
- *find(gte("paginas", 50))* - Buscará todos los documentos que tengan 50 o más páginas
- *find(and(gte("paginas", 50), lte("paginas", 75))* - Buscará todos los documentos que tengan entre 50 y 100 páginas (ambos valores incluidos)
- *find(or(eq("genero", "novela"), eq("genero", "accion"))* - Buscará todos los documentos que sean del género 'novela' y 'accion'

Ejemplos

- Devuelve el primer documento que coincida con el campo y valor especificados:

```
Document documento = db.getCollection("libros").find(eq("titulo", "Secuestrado")).first();
```

- Devuelve todos los documentos que coincidan con el campo y valor especificados:

```
FindIterable iterable = db.getCollection("libros").find(eq("titulo", "Secuestrado"));
```

- Devuelve los primeros diez documentos que coincidan con el campo y valor especificados:

```
FindIterable iterable = db.getCollection("libros").find(eq("titulo", "Secuestrado")).limit(10);
```

- Devuelve los documentos encontrados saltándose los diez primeros:

```
FindIterable iterable = db.getCollection("libros").find(eq("titulo", "Secuestrado")).skip(10);
```

- Busca en un documento que tiene otro embebido:

```
FindIterable iterable = db.getCollection("libros").find(eq("editorial.nombre", "Anaya"));
```

- Búsqueda sobre dos campos (género y número de páginas). Busca los libros de género *Novela* que tengan más de 100 páginas.

```
FindIterable iterable = db.getCollection("libros").find(and(eq("genero", "Novela"), gt("paginas", 100));
```

Operadores

A continuación, los operadores más habituales:

- Operadores de comparación: *eq, gt, gte, lt, lte, ne, in, nin*
- Operadores lógicos: *or, and, not*

> Se pueden encontrar más ejemplos de métodos para realizar búsquedas en el Tutorial de MongoDB para Java [https://docs.mongodb.com/manual/tutorial/query-documents/]

Ejercicios Examen

1. Crea una base de datos en MongoDB con tu nombre y crea una colección de juegos, almacenando la siguiente información sobre cada uno: título, género, precio y fecha de lanzamiento
    a. Inserta un juego rellenando todos los campos
    b. Inserta un juego sin indicar fecha de lanzamiento
    c. Realiza una búsqueda de todos los juegos del mismo género y ordénalos por título
    d. Crea una aplicación Java que permita conectar con la base de datos que has creado de forma que se pueda listar, en una tabla, la información de los dos juegos introducidos
    e. Añade a la aplicación la posibilidad de registrar nuevos juegos, considerando el título como campo obligatorio
    f. Añade a la funcionalidad de registrar nuevos juegos un control para que no sea posible registrar dos juegos con el mismo nombre. En ese caso se mostrará un mensaje de error al usuario y tendrá que proporcionar otro nombre diferente
    g. Añade una nueva funcionalidad que permita eliminar todos los juegos de un mismo género. Prepara un combo que liste los géneros y el usuario, seleccionando uno de ellos, podrá eliminar todos los juegos que pertenezcan al mismo
    h. Añade una funcionalidad que permita modificar los datos de un juego

---

Proyectos de ejemplo

Todos los proyectos de ejemplo de esta parte están en el repositorio java-mongodb [http://www.github.com/codeandcoke/java-mongodb] y en el repositorio de JavaFX [https://github.com/codeandcoke/java-javafx] de GitHub.

Los proyectos que se vayan haciendo en clase estarán disponibles en el repositorio datos-ejercicios [http://www.github.com/codeandcoke/datos-ejercicios], también en GitHub.

Para manejaros con Git recordad que tenéis una serie de videotutoriales en La Wiki de Entornos de Desarrollo [https://entornos-desarrollo.codeandcoke.com/apuntes:git]

Práctica 4.1

Objetivos

Desarrollar una aplicación con una base de datos NoSQL (MongoDB)

Enunciado

Siguiendo el mismo diseño de la aplicación de las práctica 1.1, 2.1 y 3.1, se deberá implementar una aplicación que conecte con una base de datos NoSQL (MongoDB), según los requisitos que se enumeran a continuación

Requisitos (1 pto cada uno)

- La Base de Datos contendrá, al menos, 2 colecciones relacionadas entre ellas.
- Se podrá llevar a cabo el alta de documentos.
- Se podrá llevar a cabo la modificación de documentos.
- Se podrá llevar a cabo la baja de documentos.
- Se podrán llevar a cabo búsquedas simples (por un campo) y complejas (utilizando condiciones para más de un campo).

Otras funcionalidades (1 pto cada una)

- Ampliar la base de datos a 4 colecciones.
- Utilizar alguna colección que contenga estructuras complejas (arrays, datos estructurados, . . .).

- Implementar un mecanismo de usuario/contraseña.
- Implementar una barra de estado donde mostrar un resumen de los datos que visualiza el usuario en cada momento (qué datos, cuantos objetos, qué objeto tiene seleccionado actualmente y los mensajes oportunos según la acción que realice).
- La aplicación refrescará los datos mostrados de forma automática antes posibles cambios en la base de datos.
- Permitir importar datos desde una base de datos relacional (MySQL, por ejemplo) a alguna colección de la aplicación.
- Permitir exportar datos a alguna tabla de una base de datos relacional (MySQL, por ejemplo).
- Permitir exportar datos de colecciones como ficheros CSV.
- Permitir importar datos de colecciones desde un fichero CSV.

© 2016-2019 Santiago Faci

1) https://docs.mongodb.com/manual/crud/ [https://docs.mongodb.com/manual/crud/]
2) FindIterable es una colección iterable para recorrer los documentos encontrados

---

apuntes/mongodb.txt · Last modified: 04/03/2021 09:20 by Santiago Faci